

Natural-Language Supervision of a 20-Robot Swarm: An LLM Orchestration Layer over Classical MAPF and Formation Control

Evgenii Shlomov Artyom Tuzov Senya Belyh Dmitry Ryabov
Dmitriy Vizitei Anastasia Malakhova

Innopolis University, Innopolis, Russia
{e.shlomov, a.tuzov, s.belykh, d.ryabov,
d.vizitei, a.malakhova}@innopolis.university

Abstract

We present `iros_llm_swarm`, a complete ROS 2 system in which a large language model supervises a 20-robot differential-drive swarm from free-form natural language. Unlike prior LLM-robotics work that emits low-level code or single-robot skill sequences, our orchestration layer emits a *typed, recursive plan tree* over four fleet-level primitives and two control containers, and every leaf is discharged by a classical algorithm — Multi-Agent Path Finding (MAPF) with two interchangeable backends (PBS and LNS2), a centralized leader-follower formation manager, and a BehaviorTree.CPP mission front-end. The LLM therefore never touches velocities, trajectories, or conflict resolution; it is confined to a small, well-typed decision surface that the underlying stack can validate, reject, and repair.

The contribution of this paper is the orchestration layer and its evaluation methodology. We describe three control channels (reactive, proactive, and operator chat), a grounding stack that gives the model bounded access to live fleet state through eight semantic tools, schema-constrained decoding against the plan grammar, and—centrally—a *defence-in-depth* design in which six deterministic guards and three LLM-in-the-loop repair loops sit between a generated plan and the robots. We further contribute a reproducible evaluation design: a 44-case hermetic planning benchmark spanning twelve structural categories, and a proximity-driven task system that turns end-to-end mission success into an automatically scored quantity, so that planning accuracy and mission completion can be measured separately and attributed to individual architectural components by ablation.

[PENDING RESULTS: This draft fixes the system description and the experimental design. E1 (hermetic planning accuracy) is populated across five local models and E2 (grounding and constraint ablation) across three; E3 remains unpopulated pending further runs.]

1 Introduction

Coordinating a fleet of mobile robots that share one workspace is a long-standing problem. The classical decomposition separates fleet-level decisions—which robot goes where, in what order—from robot-level execution—how one robot follows a

path or holds an offset. Large language models make this decomposition newly attractive [12, 13, 14, 15]: an LLM can reason over compact semantic primitives instead of producing control commands, while the classical stack retains responsibility for safety, conflict resolution, and real-time tracking.

The difficulty is that this promise is easy to state and hard to make hold. A language model asked to command twenty robots will confidently emit coordinates inside walls, address robots that are currently locked into a formation, place four robots on the same point, invent room names, and return JSON that does not parse. Any one of these silently produces a fleet-wide failure. The engineering question this paper addresses is therefore not *can an LLM plan for a swarm* but *what has to sit between the LLM and the robots so that it can*, and how much each of those pieces is actually worth.

1.1 Scope and contributions

We study a fleet of 20 differential-drive robots in a 2D environment simulated with Stage, each running its own Nav2 stack. The system spans three layers: a natural-language orchestration layer (this paper’s focus), a classical coordination substrate (MAPF, formations, behavior trees), and per-robot navigation. Our contributions:

- (1) **A typed plan language for fleet control.** Four leaf primitives (`mapf`, `formation`, `disband`, `idle`) and two containers (`sequence`, `parallel`) form a recursive tree that is expressive enough for multi-group, multi-phase missions yet small enough to be validated exhaustively and to be expressed as a JSON schema for constrained decoding (Sec. 4.2).
- (2) **Three orchestration channels** over one substrate: a reactive channel invoked by behavior-tree nodes on WARN/ERROR, a proactive channel that observes fleet state and intervenes on its own initiative, and an operator chat channel that streams a reply and an executable plan (Sec. 4.1).
- (3) **A grounding stack** that replaces coordinate hallucination with lookup: a live pose cache, a room-occupancy-aware group placement solver, and eight semantic tools brokered behind an allowlist (seven read-only), with an optional MCP provider for raw ROS introspection (Sec. 4.3).
- (4) **Defence in depth between plan and robots:** six layers—

schema-constrained decoding, a schema validator, permissive type coercion, an active-formation guard, an occupancy-aware goal rewriter, and automatic formation staging—all deterministic, none requiring another model call (Sec. 4.4). Sec. 6.2 shows the first of these is also the one that can do the most damage, which is an argument for the other four rather than against the principle.

- (5) **A bounded closed loop:** deterministic post-execution verification feeding three nested LLM repair loops (remediation, verification repair, mission supervision), each with an explicit step, time, and no-progress budget (Sec. 4.5).
- (6) **A 20-robot execution substrate** that makes the above runnable and measurable: a Stage 2D environment reached after a documented pivot from Gazebo Harmonic, five scenarios with symbolic map descriptions and a dynamic obstacle system, per-robot Nav2 stacks generated from a single parameter template with staggered bring-up, and a static-localization strategy that replaces AMCL at this fleet density (Sec. 3).
- (7) **A reproducible evaluation design** (Sec. 5) that separates planning competence from execution competence, scores end-to-end missions automatically via a proximity-driven task system, and attributes outcomes to individual components through configuration-level ablation.

The system was drafted internally as two partial write-ups organised around a three-level split of the work: one covering the classical coordination substrate, the other the simulation environment, per-robot navigation, and an early design of the orchestration layer. Each necessarily described its layer against *assumed* interfaces at the boundary. This paper is the unified account of the system as built: it supersedes both drafts, corrects the orchestration-layer description to match the shipped implementation, and—because planning quality and mission success can only be compared on the whole stack—evaluates the three layers together rather than separately.

2 Related Work

Multi-Agent Path Finding. MAPF has a rich algorithmic literature [1]. Conflict-Based Search [2] is the canonical optimal solver. Priority-Based Search (PBS) [3] relaxes optimality for scalability: a strict priority ordering replaces time-indexed constraints, and each agent plans deterministically against higher-priority paths. Large Neighborhood Search for MAPF (LNS2) [4] is anytime and suboptimal, starting from a possibly colliding plan and iteratively destroying and repairing agent subsets with soft-constraint A^* . We adopt both behind one action interface, which lets us treat the backend as an experimental factor rather than a design commitment.

Formation control. Formation control is typically posed as leader–follower, virtual structure, or behavior-based [5, 6], with potential-field and sliding-mode variants for continuous inter-robot spacing [7]. We use leader–follower because it composes cleanly with a per-robot motion controller and matches the kind of semantic offset (“a wedge”, “abreast”) a

language model naturally produces.

Navigation and behavior trees. Nav2 [8] is the de facto ROS 2 navigation stack, with layered costmaps [9] the standard obstacle representation. Behavior trees [10, 21] via BehaviorTree.CPP [11] provide a modular control architecture that is particularly suitable when an external reasoning component supplies tree parameters.

LLMs for robotics. The area is broad enough to have its own surveys [23]. SayCan [12] grounds language in affordances; Code as Policies [13] and ProgPrompt [14] emit executable program text; end-to-end vision-language-action models such as RT-2 [24] go further and regress actions directly, which trades away exactly the inspectable intermediate representation we rely on; Inner Monologue [15] closes the loop with textual feedback. Our design differs in two respects that matter at fleet scale. First, the model emits a *typed tree over fleet primitives*, not code: the output space is small enough to constrain by schema and validate exhaustively, which removes the arbitrary-code-execution surface and the malformed-output failure mode. Second, feedback is not free text but a *deterministic verification record* computed from formation status and pose state (Sec. 4.5), so the repair loop is driven by measured ground truth rather than by the model’s own narration.

ROS 2 and DDS at scale. ROS 2 [18] places DDS on the critical path, and discovery cost dominates at scale [16, 17]. We use CycloneDDS with a project-specific profile; without it the 20-robot system does not reliably complete discovery within the launch budget.

3 The Classical Substrate

This section summarizes the layer the orchestrator commands.

Shared fleet action. Both MAPF backends own the same long-lived `/swarm/set_goals` action, which encapsulates the entire *validating* $\rightarrow$ *planning* $\rightarrow$ *executing* $\rightarrow$ *replanning* $\rightarrow$ *succeeded* lifecycle behind one goal token. Schedule monitoring and replanning are internal; clients see one atomic operation. Cancellation stops every robot. The action result carries a full metrics payload—planning time, node expansions, per-agent path lengths, replan count, total execution time—which we exploit directly for benchmarking (Sec. 5.5).

Because the contract is shared, the backend is a launch-time switch and nothing above it changes: the orchestrator, the guards, the behavior tree, and the per-robot followers are all written against the action, not against a solver. E3 runs the same mission workload on both backends, which is the evidence we offer for that interchangeability. It is deliberately *not* a comparison of the two solvers—see Sec. 5.7 for why we do not make one.

PBS backend. An 8-connected A* on a grid downsampled to 0.2 m, with a backward-Dijkstra heuristic per unique goal, two-zone gradient inflation (hard footprint plus a soft penalty zone), and capsule-vs-capsule conflict detection that subsumes the usual vertex/edge split. The high-level search follows the priority-based formulation of [3].

LNS2 backend. An initial soft-constraint A* solution refined by an adaptive large-neighborhood search with four destroy operators (collision-based, random-walk, random, bottleneck) whose weights adapt online. It is anytime, tolerant of transiently infeasible starts, and warm-starts on replan.

Per-robot followers and formations. Each robot runs a dual-mode follower: in AUTONOMOUS mode it dispatches its assigned plan to Nav2 `follow_path`; in FORMATION mode it disengages Nav2 and runs a PD controller tracking $p_{\text{ref}} = p_{\text{leader}} + R(\theta_{\text{leader}})o$ for a body-frame offset o . Mode switching is driven by a single latched message, without node restarts. A centralized manager owns formation definitions and republishes the full registry on a latched topic; a monitor computes per-follower error $\varepsilon_i = \|p_i - (p_{\text{leader}} + R(\theta_{\text{leader}})o_i)\|$ and classifies each formation as INACTIVE / FORMING / STABLE / DEGRADED / BROKEN at 10 Hz. These ε_i are published, which makes formation tracking accuracy directly measurable (Sec. 5.7).

Behavior-tree front-end. BehaviorTree.CPP v3 nodes wrap every primitive: `MapfPlan`, `SetFormation`, `DisableFormation`, and a `CheckMode` guard. Mission completion is observed on the `/bt/state` blackboard mirror, whose `active_action` field cycles `none` $\rightarrow$ `node` $\rightarrow$ `none`; the orchestrator keys leaf completion on this transition rather than on the command acknowledgement, which is only an ack.

Simulation environment. The fleet runs in Stage 2D [19] via `stage_ros2`, with a unified TF tree and enforced namespace prefixes. An earlier Gazebo Harmonic [20] configuration was implemented end-to-end and then retired: per-robot physics, collision broadphase, and sensor rendering pushed the real-time factor below usable thresholds at 20 robots before any navigation stack was attached. The Gazebo configuration is retained in tree for smaller fleets and future hardware-realistic preview.

Five scenarios share one robot model and spawn protocol: `cave` (75×75 m, narrow single-file passages), `large_cave` (150×150 m, same topology at double scale), `warehouse_2` and `warehouse_4` (30×30 m, two- and four-group symmetric layouts), and `amongus` (66.6×37.75 m), which maps the spaceship layout of *Among Us: The Skeld* into 14 named rooms joined by narrow corridor choke-points. We use `amongus` for all orchestration experiments because its rich naming exercises symbolic reference and its choke-points stress conflict resolution.

Each scenario ships a machine-readable map description consumed by the orchestrator: named locations with coordinates, aliases in English and Russian, colour-coded robot groups with home and spawn positions, formation-capable zones with free-space radii, and map-specific planning heuristics (bottleneck locations, known-unreachable goal patterns, abort-versus-replan guidance). A dynamic obstacle manager additionally supports runtime-toggable doors and placeable obstacles; `amongus` declares six corridor doors matching the map geometry.

Scale engineering. Three decisions are load-bearing at 20 robots and are reported because they constrain the experiments. CycloneDDS with a custom profile is mandatory above roughly ten participants. AMCL is disabled—each robot’s scan contains every other robot’s body, and particle weights collapse—so localization is a static per-robot `map` $\rightarrow$ `odom` identity transform over simulator odometry. Bring-up is staggered (simulator 0 s, Nav2 1 s, planner 10 s, followers 12 s, plus 0.3 s per Nav2 instance).

4 Orchestration Layer

4.1 Three channels

The orchestrator exposes three independent paths from language to fleet action, which share prompt infrastructure and the plan validator but differ in trigger, latency budget, and authority.

Channel 1 (reactive). A `/llm/decision` action server. Behavior-tree nodes call it when a primitive reports `WARN` or `ERROR`, supplying the event plus a tail of the accumulated log. The model returns exactly one of `wait` / `abort` / `replan`. The prompt biases hard toward `wait` and enumerates planner-internal warnings that indicate *self-recovery in progress* (“replan did not reach collision-free; keeping old paths”, “warm seed rejected”) as explicit `wait` cases—because the expensive error here is a spurious `replan` that cancels a plan the sub-strate was about to fix on its own. Always active.

Channel 2 (proactive). A passive observer subscribing to `/bt/state`. It decides on its own initiative whether to intervene and publishes an `LlmCommand` that a behavior-tree receiver applies to the blackboard. It is cooldown-gated and checks an `llm_thinking` flag to avoid colliding with channel 1. Off by default.

Channel 3 (operator chat). The main path, and the focus of this paper. An operator types free-form text; the server streams a natural-language reply token-by-token, parses an executable plan from the same response, and dispatches it. Everything in Secs. 4.2–4.5 describes channel 3.

4.2 Plan language

Channel 3 returns a single JSON object `{reasoning?, reply, plan}`, where `plan` is a recursive tree over the node types in Table 1.

Table 1: Plan tree node types.

Node	Kind	Payload / semantics
mapf	leaf	robot_ids, goals, spread; one goal per robot, or one centre that the server grids
formation	leaf	formation_id, leader_ns, follower_ns, offsets_x/y
disband	leaf	formation_id
idle	leaf	reason; also the carrier for escalation (see below)
sequence	container	children in order; abort on first failure
parallel	container	see flattening rules below

Three design decisions in this grammar are worth stating explicitly.

Parallelism is a claim about intent, not about threads. A `parallel` node is flattened before execution: multiple `mapf` leaves *merge into one* `mapf` goal, because the MAPF planner already resolves inter-robot conflict internally and issuing two concurrent fleet goals would defeat exactly the coordination we want. Mixed `mapf` and `formation` children degrade to sequential execution, since a formation requires its own follower mode. An `idle` in any branch short-circuits the whole node—stop takes priority. This means the operator’s “do A and B at the same time” is honoured as *one jointly planned motion*, which is stronger than concurrent dispatch, not weaker.

idle doubles as the escalation channel. Rather than adding node types for “I need clarification” and “here is an answer, no action needed”, the model emits `idle` with a prefixed reason: `needs_help:`, `clarify:`, or `reply_only:`. This keeps the grammar at six node types—small enough to constrain by schema—while still distinguishing refusal-to-act from action. It also makes escalation a first-class, testable behaviour: a benchmark case can assert that a question with a missing robot pose produces `needs_help:` and not a confident plan over stale data.

The schema is enforced twice. `plan_schema.py` expresses the grammar as a JSON schema with a recursive `$defs/node` reference, passed to the backend for constrained decoding (Ollama format, OpenAI `response_format`). But JSON Schema cannot express the cross-field constraint `|goals| = |robot_ids|`, so a hand-written validator re-checks every node after parsing. Constrained decoding removes the *malformed* failure mode; the validator removes the *incoherent* one.

4.3 Grounding

A model asked for “cyan to the cafeteria” must resolve a colour group to robot ids, a room name to coordinates, and coordinates to free floor space. We push all three out of the model’s head and into lookup.

Static context. The prompt carries a per-map description: named locations with coordinates, colour groups with member ids and home positions, formation-capable zones

with radii, and door geometry. Maps are declared in `common_scenarios.yaml`, so adding an environment does not touch the orchestrator.

Live context. A pose cache subscribes to `/robot_*/odom` and supplies fresh positions with an explicit staleness bound (2 s). This is what lets the model compute formation goals as *leader_{xy} + offsets* rather than inventing absolute coordinates—and, equally important, lets it *notice a missing robot* and escalate instead of planning over a gap. Formation status, behavior-tree state, recent events, and open task state are folded into the same snapshot under a character budget.

Semantic tools. Eight tools sit above raw introspection. `get_robot_position` returns one robot’s live map-frame pose; `get_positions` resolves named geometric points such as room corners, doorways and choke-points; `check_occupancy` returns the local occupancy grid around a robot, built from its lidar. `find_free_group_goals_in_room` computes deterministic occupancy-aware per-robot goals for ordinary group movement, and `find_group_placement_in_room` the non-overlapping leader/follower placements a formation needs. `verify_plan_execution_state` checks post-execution formation state and follower offset errors. `list_tasks` and `reset_task` read and write the task ledger. They are dispatched by a broker that enforces an allowlist, a per-round tool cap, a timeout, and a result-size cap. An optional MCP provider (`ros-mcp` over `stdio`) adds raw ROS introspection under a separate eleven-verb allowlist restricted to reads. The model never sees MCP tools directly—only a bounded snapshot or a brokered semantic call.

Seven of the eight are read-only; `reset_task` is the exception, and it is worth naming rather than glossing. It returns a task to `pending` so a stuck or mis-scored task can be re-tried, which mutates the same task state that Sec. 4.6 uses to score missions. The write surface is therefore narrow and confined to the task ledger—no tool commands a robot—but it is not empty, and an evaluation that scores itself from task state should not expose a tool that can rewrite it. We flag this as a defect of the current allowlist rather than a property of the design.

This is the learned-tool-use pattern of Toolformer [22] applied under a broker rather than in the open: the model chooses among a fixed, audited set instead of inventing calls.

Notably, tool calling and schema-constrained decoding are *mutually exclusive* in current backends: constrained decoding suppresses tool-call emission. The system therefore has two distinct planning modes, which we treat as a primary experimental factor (Sec. 5.4).

4.4 Defence in depth

Table 2 lists what stands between a generated plan and the robots. The design principle is that *a deterministic guard is always preferred to another model call*: guards are cheap, reproducible, and cannot themselves hallucinate.

Two of these deserve elaboration because they encode domain knowledge the model does not reliably have.

Table 2: Layers between a generated plan and the fleet. D = deterministic, L = requires an LLM call.

Layer		Failure mode addressed
Constrained decoding	D	unparseable / off-grammar JSON
Schema validator	D	structurally valid but incoherent nodes
Type coercion	D	"robot_10" where an <code>int</code> is required
Active-formation guard	D	direct MAPF control of a robot locked in a live formation
Occupancy rewrite	D	room-centre goals that land in walls or clump
Formation prestaging	D	bare <code>formation</code> leaf with followers out of position
Remediation loop	L	execution failed; replan with failure briefing
Verification repair	L	execution reported success but the outcome does not hold
Mission supervision	L	multi-step mission needs continuation

Type coercion is deliberately permissive. Small local models routinely emit "robot_10" or 10.0 where the grammar wants 10. A strict parser would reject an otherwise perfect plan over a namespace string. The coercer accepts 10, "10", "robot_10", "robot10", and integral floats, while explicitly rejecting booleans (which are `int` subclasses in Python and would otherwise silently become robot 0 or 1). Strictness is spent where it buys safety, not where it only punishes formatting.

The *active-formation guard* exists because "send the magenta line to storage" is ambiguous in a way that matters. A formation is commanded by moving its *leader*; commanding all four members individually tears it apart. The guard reads live formation status, and for any formation in FORMING, STABLE, or DEGRADED it strips follower ids out of MAPF leaves, leaving the leader. It returns a structured failure record shaped exactly like a PlanExecutor failure, so the existing repair loop consumes it with no special handling.

4.5 Closed loop

Execution success is not the same as mission success, and this distinction drives the whole loop design. A plan can execute cleanly—every action server returns success—while the requested outcome does not hold: robots arrive but the formation never stabilizes, or they arrive at a room’s centre and pile up. The system therefore verifies deterministically after execution, reading formation status and pose state to decide whether what was asked for is actually true, and emits an honest *partial* result when state is missing rather than guessing.

Three nested loops consume that verdict, each explicitly budgeted:

- **Remediation** (default 2 attempts) fires when execution *fails*. It re-prompts with the failed leaf, the error, the history of prior attempts, and a *targeted refresh* of runtime context—not the full snapshot, to keep the retry prompt inside the context budget.
- **Verification repair** (default 2 attempts) fires when execution *succeeds* but verification says the outcome does not hold, and only when the verifier itself marks the discrepancy repairable.
- **Mission supervision** (default 6 steps, 180 s) wraps both in a bounded execute–verify–continue loop for multi-phase missions, with a no-progress limit of 2 and a fingerprint set of already-failed plans so the model cannot loop on a plan that has already been tried.

Every loop terminates in a help escalation to the operator rather than in silent failure or unbounded retry. We regard these budgets as tuning parameters, not constants, and measure their effect directly (Sec. 5.5).

4.6 Task system as mission ground truth

A separate node maintains a set of spatial tasks and polls TF at 5 Hz. A *point* task transitions PENDING → DONE when any robot enters its radius. A *carry* task transitions PENDING → CARRYING on pickup and → DONE when the carrying robot reaches the dropoff. Tasks are declared per scenario in YAML and their state is published.

This component was built for mission planning, but it doubles as the measurement instrument that makes end-to-end evaluation tractable: it converts “did the swarm accomplish what the operator asked” into an automatically scored, timestamped, operator-independent quantity. Every end-to-end experiment below is scored from this signal.

5 Experimental Design

This section is the methodological core of the paper. We state it in full before reporting numbers so that the measurement protocol is fixed independently of its outcome.

5.1 Research questions

- RQ1** How accurately does a locally-hosted open-weight LLM translate free-form fleet commands into valid plans, and how does accuracy vary across linguistic and structural categories?
- RQ2** What does each grounding and constraint mechanism contribute? Specifically: schema-constrained decoding vs. tool calling, live pose context, and context budget. *Partially answered:* Sec. 6.2 reports the constraint and grounding halves; the comparison against tool calling is deferred to E3 for the reason given there, so RQ2 is not closed by this draft.
- RQ3** Does planning accuracy predict mission success, or does the gap between them—the part closed by guards and repair loops—dominate?

Table 3: Metrics and their existing sources.

Quantity	Source (already implemented)
Plan validity, category accuracy, latency	44-case benchmark harness
MAPF planning time, expansions, path lengths, replans	SetGoals action <i>result</i> payload
Arrival / deviation / stall counts	SetGoals action <i>feedback</i>
Formation error ϵ_i , state machine	FormationStatus @ 10Hz
Mission completion, time-to-complete	TaskState from the task manager
Repair / remediation / supervision counts	channel-3 event stream

RQ4 What does each closed-loop layer contribute to end-to-end mission completion, and at what cost in time and model calls?

An earlier version of this design carried a fifth question—which of the two MAPF backends is better under identical fleet requests. We have withdrawn it as a research question rather than answered it narrowly. Sec. 5.7 gives the reason: the comparison is confounded on this system in ways parameter alignment cannot fix, and the residue that *is* measurable—that the orchestration layer completes the same missions on either backend—is a property of the action contract described in Sec. 3, not a question this paper investigates. We report it there and in E3, and claim nothing about which backend is superior.

RQ3 is the question we consider most important and the one the literature most often skips: a benchmark measured on plan text alone can look excellent while the fleet still fails, and the converse—weak plans rescued by strong guards—is equally possible. Measuring both on the same system is the point.

5.2 Principle: reuse existing instrumentation

A deliberate constraint on this design is that it introduces almost no new measurement code. Every metric below is read from a signal the running system already produces (Table 3). This matters for validity: an evaluation harness that reimplements the notion of success is measuring the harness, and instrumentation added for a paper tends not to survive the paper.

5.3 E1: Planning accuracy (hermetic, offline)

The 44-case benchmark calls the model directly—no simulator, no ROS—with *fixed synthetic runtime contexts* (fleet at spawn; a stable magenta line; a stable cyan wedge; robot 12’s pose missing; robot 4 at the cafeteria). Each case pairs a prompt with a programmatic validator asserting structural properties of the returned tree: node type, exact robot-id sets, goal proximity to a named location within a tolerance, goal distinctness, formation leader identity, staging-before-formation ordering, and escalation-reason prefixes.

Cases span twelve categories chosen to separate distinct competences: basic single-group `mapf`; explicit per-robot coordinates; goal spreading; `parallel` over multiple groups; multi-phase `sequence`; formation creation (which requires *staging followers first and excluding the leader*); formation movement (which requires commanding *only* the leader); `formation_complex` (creation followed by a leader-only move); `disband`; `idle`; escalation under missing or ambiguous state; and a set of edge cases (repeated/ambiguous commands, partial formations, adjacency requests). All prompts here are English. An earlier revision of this benchmark additionally scored reply-language fidelity on a parallel set of Russian-language prompts; we dropped that set because it tests a UI-level concern (does the chat reply mirror the operator’s language) orthogonal to planning accuracy, and folding it into a single pass rate obscured both signals.

Protocol. Each case is run $N=5$ times per model at temperature 0.1. We report `pass@1` (mean over repeats) and `pass@5`, per-category breakdown, end-to-end latency, and the rate of each distinct failure mode (unparseable, schema-invalid, wrong ids, wrong coordinates, wrong structure). All runs reported here use `-llm-max-tokens 2048` (raised from an initial 768, which was tight enough to make truncated JSON a confound for more verbose models independent of their planning ability) and `-llm-num-ctx 32768`.

Models. The deployment target is a single workstation with 16 GB VRAM, which fixes the candidate set. The prompt is large—we measured $\approx 9,700$ tokens for the system prompt plus seven few-shot exchanges, before runtime context—so KV-cache cost is a real constraint, not a rounding error. We sweep five locally-hosted instruction models spanning that budget: a small model (**qwen3.5:4b**, 3.4 GB); a mid-size model from a different family (**gemma3:12b**, 8.1 GB); the intended 4-bit 14B operating point (**qwen2.5:14b**, 9.0 GB); an 8-bit ≈ 9 B model (**qwen3.5-9b**, 10 GB); and the largest model that still fits under the VRAM ceiling (**mistral-small3.2:24b**, 15 GB, 4-bit, partial CPU offload expected). Whether the last of these remains practical under offload is itself part of what the latency column reports.

“Intended operating point” above means the size class we targeted when the sweep was designed, and it is worth separating from two other senses of “the deployed model”, because the three do not currently agree. `orchestrator.yaml` ships `mistral-small3.1`—a different model from any swept here, chosen before these measurements existed and never revisited. E1 then selects **qwen3.5-9b** on both pass metrics, which is neither the configured default nor the 14B size class we set out to use. We report the sweep rather than a single deployment because of exactly this: the configuration file is not evidence about which model is best, and until E3 runs end-to-end we treat none of the three as settled. Sec. 6.2 shows why the gap matters beyond bookkeeping—the schema finding is model-specific, so a stale default is not a harmless one.

Threat. E1 measures planning in isolation against synthetic context. It deliberately cannot detect a plan that is structurally perfect and physically unexecutable. That is E3’s job, and the comparison between them answers RQ3.

5.4 E2: Grounding and constraint ablation

Same harness, same cases, toggling one mechanism at a time: schema-constrained decoding on/off; live pose context on/off; goal auto-spread on/off; and context budget swept across its range. These are runtime parameters of the chat server rather than of the offline harness, and Sec. 6.2 records which of them turned out to be reachable from the harness at all.

We add one factor that is not currently a parameter: *symbolic versus raw grounding*. During development we observed that prompting with raw coordinates produced a higher rate of out-of-bounds and wall-interior goals than symbolic room references, and that supplying named locations, formation zones, and map-specific heuristics substantially reduced goal-validation failures downstream. That observation was uncontrolled and retrospective. Because the map description is a separable prompt block, it can be ablated cleanly—full description, locations without heuristics, and coordinates only—turning an anecdote into a measured effect size. Given that the map description is a large share of a $\approx 9,700$ -token prompt, this also quantifies what that context is buying.

One asymmetry must be stated plainly. The offline harness invokes the model directly and therefore exercises only the plain and schema-constrained paths; the *tool-calling* path lives in the chat server. Comparing constrained decoding against tool calling—the mutual exclusivity noted in Sec. 4.3, and arguably the single most interesting ablation here—therefore requires either extending the harness to route through the chat server, or measuring that factor only in E3. We initially chose the first, on the grounds that the second confounds planning mode with execution noise. In the event we took the second: as reported in Sec. 6.2, too little of the tool surface is meaningful offline for the comparison to be worth its apparatus, and the confound we accept in exchange is stated there.

5.5 E3: End-to-end missions

The primary experiment. A 20-robot stack runs in the 14-location environment with nine declared tasks (six point, three carry). An operator command is issued through channel 3; the system plans, executes, verifies, and repairs; the task manager scores completion independently.

Factors. Remediation on/off; verification repair on/off; mission supervision on/off; planning mode (constrained vs. tool-calling); model size; MAPF backend. The planning-mode factor must be read together with Sec. 6.2: on a model that alphabetises keys in schema mode, the constrained arm reproduces the E2 collapse and E3 would measure that rather than planning mode. The factor is therefore only interpretable with the model held fixed to one E2 shows is unaffected.

Mid-mission environment change. Two cells repeat the reference mission and close one door of the environment sixty seconds in, chosen so that each breaks exactly one leg of the plan while the other four keep running as an in-run control. The first closure lengthens a carry leg from 38 m to 75 m: the route is invalidated but an alternative exists. The second disconnects the target of another leg outright: no path to it re-

mains at all. This is the case the repair loops of Sec. 4.5 are justified by—the world changing after a plan is committed—and every other cell exercises them on execution failures instead. The sharpest quantity is not completion but escalation: in the disconnected cell, confidently re-planning the same unreachable goal is an unambiguous failure, while reporting that it cannot be done, or completing the remaining legs and saying so, is correct. That directly extends the E1 finding that escalation judgment is weak and flat across model sizes (Sec. 6.1), by testing it on a live fleet rather than on plan text. The door state is visible to the model in its per-turn context, so a failure here is not a failure of information.

Metrics. Task completion rate; time to completion; number of LLM calls per mission (the cost of each safety net); how often each guard fired; how often each repair loop fired and whether it succeeded; escalation rate; and total fleet distance travelled.

Protocol. N repeats per configuration with a fixed initial fleet state, resetting task state between runs via the existing reset service. Because a 20-robot stack is stochastic—DDS timing, Nav2 controller noise—single runs are not informative, and we report distributions, not point estimates. This directly addresses a limitation the earlier internal drafts acknowledged: their integration scenario was run once.

No-LLM baseline. The workspace includes a scripted behavior-tree runner that drives the identical six-step mission—fleet convergence, two formation activations, a full diagonal cross-swap, return home, idle—against the same live 20-robot stack with no LLM in the loop. This is the natural ceiling for E3: it isolates what the substrate achieves when plans are known-correct by construction. The difference between scripted completion and LLM-driven completion is precisely the orchestration cost, and reporting E3 without it would leave the reader unable to tell a weak orchestrator from a hard environment.

Guard attribution. Because guards are deterministic, their contribution is measurable without disabling them: we log every firing and, for each, whether the plan would have been physically invalid without it. Disabling a safety guard on a live 20-robot fleet to measure it is neither necessary nor wise.

5.6 E3b: Channel-1 decision accuracy

Channels 1 and 2 are the least evaluated part of the system in either report, and unlike channel 3 they already write structured per-call JSONL logs (prompt, raw output, parsed verdict). This makes a replay evaluation nearly free: accumulate incident records across the E3 campaign, have two annotators independently label the correct verdict (*wait/abort/replan*) from the event and log tail, and score the model against that label set with inter-annotator agreement reported.

The interesting quantity here is not overall accuracy but the *asymmetric* error rate. The prompt deliberately biases toward *wait*, because a spurious *replan* cancels a plan the substrate was about to repair on its own, whereas a spurious *wait* merely costs one supervision cycle. We therefore report the confusion matrix rather than a scalar, and separately the false-

replan rate on the planner-internal warnings the prompt enumerates as self-recovery cases.

5.7 Two experiments folded into E3, and why

Earlier versions of this design carried two further standalone experiments: a controlled PBS-vs-LNS2 comparison across fleet size and clutter, and a formation-tracking study across shape, formation size, and leader speed. Both are withdrawn as standalone experiments; each survives as one cell of the E3 grid. Because a withdrawal is itself a claim about what can be measured, we state the reasoning rather than quietly dropping the sections.

The backend comparison is confounded in ways parameter alignment cannot fix. Most differences between the two backends can be equalized by configuration—grid resolution, planning timestep, arrival tolerance, replan cooldown, low-level expansion budgets—and we would have. Three cannot. First, the two solvers terminate on different criteria: LNS2 is bounded by a wall-clock repair budget and must return the best incumbent it has, whereas PBS is bounded by high-level node expansions and is not time-bounded at all, so LNS2’s planning time is capped by construction and a head-to-head reading of it is not meaningful. Second, the collision models differ in kind, not degree: PBS keeps a hard footprint with a soft gradient penalty zone around it, while LNS2 adds its inflation directly to the footprint as a hard radius—identically named parameters with different semantics, which equalizing numerically would hide rather than remove. Third, the two backends ship with different path-following stacks, so every execution-side metric measures a planner-follower pair. A comparison published under those three caveats would say less than the space it occupies, in a paper whose central question (RQ3) does not depend on it.

What remains is the claim we actually make: the two backends are *interchangeable* behind one action contract (Sec. 3). E3 runs the full mission workload on both, which is evidence for that and for nothing more. We deliberately do not restate it as a research question: an outcome of “the same” would confirm what the architecture already asserts, and an outcome of “different” would not be attributable, given the three confounds above. A question whose answers are respectively redundant and uninterpretable is a system property, and we report it as one.

Formation tracking becomes a characterization rather than a study. One E3 mission forms a column, traverses a 21 m corridor that narrows to 1.30 m, and disbands. The formation monitor already publishes ε_i at 10 Hz and E3 already records that topic, so steady-state error, peak error, time-to-STABLE, and DEGRADED dwell fraction come out of a recording we take anyway. That mission is also the only one that exercises two of the six guards—automatic formation staging and the active-formation guard—which would otherwise report zero firings for the whole campaign for want of a trigger. What is lost is the dependence of tracking error on shape, size, and speed: we report one column of four at one speed, and we report it as a property of this system rather than as a study of

formation control.

5.8 Instrumentation to be added

Three small additions are required, and we note them explicitly because they are the only new code the evaluation needs:

- (i) A per-mission JSONL logger on channel 3. Channels 1 and 2 already write structured logs through a shared writer; channel 3 does not, and E3 cannot be scored without it.
- (ii) Repeat and machine-readable-output flags on the benchmark harness, which originally reported only to the terminal and an exit code. *Added:* the `-repeat` and `-json` flags used by E1 and E2, together with the `-grounding`, `-schema-constrained` and `-schema-variant` factors and a prompt-fingerprint check for baseline reuse.
- (iii) A bag-based offline scorer for E3, in preference to a live scoring node, so that runs can be re-analysed without re-execution.

5.9 Threats to validity

Simulation only. All results are in Stage 2D with simulator odometry as ground truth and AMCL disabled. Localization error, wheel slip, and real sensor noise are absent. Conclusions about orchestration transfer more readily than conclusions about tracking accuracy.

The simulator pivot is qualitative. The move from Gazebo to Stage was driven by observed infeasibility at 20 robots, not by a controlled throughput sweep. We report real-time factor as a function of fleet size and navigation load for the Stage configuration only, and make no quantitative claim about the crossover point. The same applies to bring-up reliability: staggered launch is observed to work, but failure rate as a function of stagger interval and DDS profile has not been measured over repeated cold boots.

Single host. Twenty robots on one workstation is a DDS-bound ceiling; timing metrics are contaminated by host load and should be read as relative, not absolute.

Benchmark construction. The 44 cases and the prompt were developed together by the same authors, which risks the benchmark encoding the prompt’s idiosyncrasies. The category breakdown and the E1/E3 comparison partially expose this, but we do not claim the benchmark is unbiased with respect to our own system. An independent case set would be a genuine improvement.

Model non-stationarity. Hosted models change under fixed names. We pin local weights and report quantization for every reported run.

Environment change is tested by route invalidation, not agent loss. The two door cells break the map while every robot keeps working. The complementary failure—a robot dies while the route stays valid—is not covered, and repair behaviour may differ between the two: a closed door is visible in the model’s context, whereas a dead robot must be inferred from the absence of motion.

Table 4: E1 overall pass rates and latency (44 cases, $N=5$, 220 calls/model).

Model	pass@1	pass@5	med. (s)	p90 (s)	t/o
qwen3.5:4b	68.2%	88.6%	4.8	8.6	0
gemma3:12b	68.2%	70.5%	3.7	7.4	0
qwen2.5:14b	72.7%	79.5%	9.3	20.1	0
qwen3.5-9b	77.3%	90.9%	8.2	21.0	0
mistral-small3.2:24b	75.0%	77.3%	29.9	52.1	0

6 Results

6.1 E1: Planning accuracy

Table 4 covers all five models swept (Sec. 5.3): **qwen3.5:4b** (small), **gemma3:12b** (mid-size, different family), **qwen2.5:14b** (4-bit, the intended operating point), **qwen3.5-9b** (8-bit), and **mistral-small3.2:24b** (4-bit, largest that fits). All runs use $N=5$ repeats per case (44 cases $\times$ 5 = 220 calls per model), temperature 0.1, `-llm-max-tokens 2048`, `-llm-num-ctx 32768`.

Raising the token budget from 768 to 2048 (Sec. 5.3) removed timeouts entirely across all five models. The best model on both metrics, qwen3.5-9b, is not the largest: parameter count does not predict accuracy here. Two further patterns stand out. First, the *smallest* model, qwen3.5:4b, is close behind on pass@1 and second-best on pass@5 (88.6%)—the gap between its pass@1 and pass@5 is the largest in the table, meaning most of its failures do not repeat under resampling and are closer to sampling noise than to a structural gap. Second, gemma3:12b and mistral-small3.2:24b show almost no such recovery (68.2% $\rightarrow$ 70.5% and 75.0% $\rightarrow$ 77.3%): their failures are largely the *same* failure on every retry, i.e. systematic rather than stochastic. Latency also does not track accuracy: mistral-small3.2:24b’s weights (15 GB, 4-bit) leave under 1 GB of headroom on the 16 GB card, forcing partial CPU offload and a median latency $6\times$ qwen3.5-9b’s—without a corresponding accuracy gain.

The five models fail in qualitatively different ways (Table 5). *Wrong formation fields*—almost entirely a single validator complaint, `follower_ns(N) != offsets_x(M) or offsets_y(M)`, i.e. the model emits an offsets array shorter than its own follower list—is the single largest failure mode for three of five models, and for mistral-small3.2:24b it is byte-identical across 25 of its 57 failures. That repetition, combined with the model’s near-zero pass@1 $\rightarrow$ pass@5 recovery, is the signature of a deterministic generation bug in how this model fills the formation-node schema, not of noisy sampling. *Wrong structure* (choosing the wrong top-level node type, e.g. `mapf` instead of `sequence`) dominates for qwen2.5:14b (28/60 failures) and gemma3:12b (25/68); qwen3.5:4b has no single dominant mode, consistent with its noise-like failure profile above. *Wrong escalation reason* is essentially constant across four of the five models (≈ 10 failures each, qwen2.5:14b is the outlier at 5), suggesting that judging when to ask for help rather than act is uniformly hard and does not improve much with scale or family within this range—

Table 5: E1 failure-mode counts (failed calls only, out of 220 each).

Failure mode	qwen3.5 4b	gemma3 12b	qwen2.5 14b	qwen3.5 9b	mistral sm3.2 24b
Unparseable JSON	16	1	0	6	11
Wrong structure	17	25	28	11	0
Wrong robot-id set	0	0	0	0	1
Wrong goals	11	0	10	8	5
Wrong formation fields	22	24	9	2	25
Wrong escalation reason	10	10	5	10	10
Other validator	0	8	8	1	5
Total failed / 220	76	68	60	38	57

an argument for a deterministic escalation check outside the model rather than prompting harder.

By category, all five models are reliable (100% pass@1) on `idle` and `parallel`, and near-reliable on `mapf_basic` and `sequence`. `formation_create` shows the widest spread across models (100% for qwen3.5:4b down to 20% for gemma3:12b), consistent with the `formation-schema` failure mode above being model-specific rather than a shared difficulty. One category is weak for *every* model regardless of size or family: `mapf_spread` sits at exactly 50% pass@1 for all five—worth flagging as a possible benchmark-design artifact rather than a genuine model weakness, since the category has only two cases and a single flipped call moves the rate by 50 points; we do not read anything into it beyond that caveat. `escalate` ranges 25–75% pass@1 with no relationship to model size (qwen2.5:14b, the largest of the mid-size models, is the *worst* on this category), reinforcing the failure-mode-table observation that escalation judgment does not scale predictably.

Reading with appropriate caution. $N=5$ per case, and as few as 2 cases in some categories, is enough to separate the reliable categories from the weak ones but not enough to bound per-category confidence intervals tightly; the category-level percentages above should be read as pointing at where the failures cluster, not as precise estimates. We report them because E2 and E3 (Secs. 5.4, 5.5) are designed to explain *why* those specific categories fail, not merely to confirm that they do.

6.2 E2: Grounding and constraint ablation

E2 toggles one mechanism at a time against the E1 configuration, on the two qwen3.5 models: the smallest (**qwen3.5:4b**) and the most accurate (**qwen3.5-9b**). Two factors are measured: schema-constrained decoding on/off, and the amount of symbolic scaffolding in the map block (*full* $\rightarrow$ *locations-only*, dropping the heuristics paragraph $\rightarrow$ *coords-only*, additionally dropping the map prose, location and group aliases, and formation zones; 3,336 $\rightarrow$ 1,795 $\rightarrow$ 1,024 characters). **qwen2.5:14b** is added for the schema factor only, as a third data point from a different family and the intended 4-bit operating point of Sec. 5.3. The schema-off, grounding-full cell *is* the E1 configuration and is reused rather than re-run; that reuse was gated on a SHA-256 over all 44 assembled prompts matching between the pre-ablation and shipped code, checked

Table 6: E2 ablation: pass rates by condition (44 cases, $N=5$). Δ is against that model’s baseline. Only the schema factor was run for qwen2.5:14b. *unp.* is the *unparseable JSON* failure count over all 220 calls—the failure mode the schema exists to remove.

Model	Condition	pass@1	Δ	pass@5	Δ	unp.
qwen3.5 4b	baseline (E1)	68.2%		88.6%		16
	schema on	72.7%	+4.5	90.9%	+2.3	1
	locations-only	70.5%	+2.3	86.4%	−2.2	11
	coords-only	63.6%	−4.6	79.5%	−9.1	11
qwen2.5 14b	baseline (E1)	72.7%		79.5%		0
	schema on	70.5%	−2.2	79.5%	±0.0	0
qwen3.5 9b	baseline (E1)	77.3%		90.9%		6
	schema on	11.4%	−65.9	18.2%	−72.7	3
	locations-only	86.4%	+9.1	90.9%	±0.0	1
	coords-only	86.4%	+9.1	90.9%	±0.0	0

automatically before each sweep rather than assumed. Protocol is otherwise E1’s: $N=5$, temperature 0.1, 2048-token budget, 32,768-token context, 220 calls per cell, 0 timeouts in all seven new ablation runs. The schema-ladder runs of Table 7 are additional to these seven and are counted separately.

Schema-constrained decoding buys nothing it was not already needed for, and reverses sign on one model. Its purpose is to remove malformed JSON, and the *unp.* column shows it doing exactly that where the problem exists: qwen3.5:4b falls from 16 unparseable calls to 1, and both its pass rates improve modestly. qwen2.5:14b never emitted malformed JSON to begin with ($0 \rightarrow 0$), so the schema has nothing to fix and changes nothing—pass@5 is identical, pass@1 moves -2.2 , and its failure-mode counts barely shift (*wrong structure* 28 $\rightarrow$ 26, the rest within one). For qwen3.5-9b the same mechanism is catastrophic: pass@1 drops 65.9 points and pass@5 72.7, while repairing only three malformed calls.

The collapse is deterministic rather than stochastic. pass@5 recovers only to 18.2%, so resampling five times almost never rescues a failed case, and the per-category breakdown is degenerate: 0.0/0.0 on eight of twelve categories—*mapf_basic*, *mapf_explicit*, *mapf_spread*, *parallel*, *sequence*, *formation_create*, *formation_move*, *formation_complex*—while scoring 100/100 on *idle*. *Wrong structure* rises from 11 to 131 and *wrong robot-id set* from 0 to 32 (cf. Table 5).

The mechanism is a key-order inversion, and it is measurable. The common reading of such a collapse—that the plan degenerates to the simplest branch of the *oneOf*—is not what the responses show. Counting the top-level node type actually emitted, *idle* accounts for 104 of 220 calls but *sequence* and *parallel* together account for 109, so *idle* does not dominate. What happens is narrower: two of the six branches become unmittable. *mapf* goes $58 \rightarrow 0$ at the top level and *formation* $7 \rightarrow 0$; counting every nesting depth, *formation* is never emitted once in 220 constrained calls and *mapf* falls $268 \rightarrow 53$, surviving only nested inside a *sequence*. This also rules out recursion as the cause, since *sequence* and *parallel* are the only branches that use the recursive *\$ref* and they are untouched.

The cause is visible in key order. Unconstrained, qwen3.5-9b emits *type* first in every object; under the schema it emits every object with its keys in strict alphabetical order, in 220 of 220 nodes. Because *type* sorts last in all six branches, the model must commit to a branch by emitting a *content* key before it has named the node type, and branch selection falls to whichever required key sorts first. That key is *reason* for *idle*, *steps* for *sequence/parallel* and *formation_id* for *disband*—and the observed first-key histogram matches the surviving branches exactly (*reason* 104, *steps* 109, *formation_id* 4). For the two branches that vanish it is *goals* and *follower_ns*, which the model never selects to open a plan object.

One step of this is correlational rather than causal, and we mark it as open. Alphabetical ordering explains why the discriminator arrives last, and hence why the branch must be committed on a content key; it does not by itself explain why *goals* is chosen *exactly zero* times in 220 calls. A mere re-ordering should depress that probability, not annihilate it. The natural conjecture is that an opening “*goals*” token has essentially no mass in the model’s distribution over plan-object prefixes—in ordinary JSON an array-valued content field does not begin an object—so the constrained sampler renormalises over the alternatives that do, and the absorbing state is reached on the first token rather than accumulated over the object. We have not tested that, and a token-level probe of the constrained sampler would settle it.

The inversion also corrects what the free text shows. It is the *reason* field inside the degenerate plan that preserves the request—echoing at least 60% of the prompt’s content words in 57% of idle responses, against 41% for *reply*—while *reply* confabulates a justification for idling (“send cyan to cafeteria” $\rightarrow$ *reason* “cyan to cafeteria”, *reply* “Stopping all robots.”). This follows from the same reordering: unconstrained the model emits *reply* then *plan*, and under the schema *plan* then *reply*, so it commits to a node type and then rationalises it. The model is not prevented from expressing an answer it knows; it records the intent, commits to the wrong branch, and justifies the result.

A schema ladder isolates the responsible construct. To separate the *oneOf* from the recursion it contains, we re-ran qwen3.5-9b under three derived schemas: **S2**, the shipped schema minus the *sequence* and *parallel* branches (four-branch *oneOf*, no *\$ref*, no *\$defs*); **S3**, one schema per node type with *type* pinned by a single *const* and no *oneOf* at all; and **S1-no-idle**, the shipped schema minus its cheapest branch. Each variant is derived programmatically from *PLAN_RESPONSE_SCHEMA* and asserted at import to reproduce it exactly when all six branches are kept, so the ladder cannot drift from the schema it is meant to explain.

Restricted schemas cannot express every reference answer, and scoring them over all 44 cases would measure “*the grammar forbids the right answer*” rather than “*the grammar breaks the model*”. Each variant is therefore scored only on the cases it can express, with the baseline and S1 recomputed on the same subset; Table 7 reports subset sizes, and none of its figures are comparable to the full-44 numbers in Table 6.

Table 7: Schema ladder on qwen3.5-9b, scored on expressible subsets only (pass@1/pass@5, $N=5$). *Not* comparable to Table 6. S1-no-idle applies only to the mapf subset. [†]S3 pins the node type, which the validators also check—its rate is *not* like-for-like with the others and is reported only to show that the branch body is reachable.

Variant	Plan node	23-case subset	13-case mapf subset
schema off	—	73.9/82.6	76.9/84.6
S1	6-branch, recursive	21.7/30.4	0.0/0.0
S2	4-branch, no recursion	26.1/34.8	0.0/0.0
S3 [†]	1 branch, no oneOf	78.3/87.0	84.6/92.3
S1-no-idle	5-branch, recursive	—	0.0/0.0

Three readings follow, and they agree with the emitted-node evidence above. *Recursion is not the cause:* S1 and S2 are both exactly 0.0/0.0 on the mapf subset, so deleting the recursive branches and every \$ref changes nothing. Their difference on the 23-case subset (21.7/30.4 vs. 26.1/34.8) is one to two cases at $N=5$ and we read it as noise, not as a recursion effect in the opposite direction. *The oneOf is:* pinning type to a single const restores mapf in 65 of 65 calls, against 0 of 65 under both S1 and S2. *And the collapse is not simply a flight to the cheapest branch:* removing idle does not bring mapf back, it relocates the degeneracy to sequence (43 of 65) and disband (15), with mapf still at exactly zero. What the model cannot do is *select* the branch; filling it was never the problem.

S3’s *pass rate* must not be read as a benefit of constrained decoding, and we flag this because the comparison is inviting and wrong. S3 pins the node type by const, and node type is exactly what the validators check first; *wrong structure* is qwen3.5-9b’s dominant failure mode under the schema (11 → 131). On the mapf subset S3 therefore tells the model the answer to the question it is being scored on, and its 84.6/92.3 is not comparable to the unconstrained 76.9/84.6—one condition has half the task solved for it in advance. What S3 licenses is the structural claim only: the mapf branch is reachable, its body is filled correctly when reached, and the failure is located in branch *selection*. Whether constraining decoding helps a model that must still choose is not something this ladder measures.

Scope. Diagnostics place the trigger in the model, not in this stack or this schema. No chat template injects the schema into the prompt, nothing on the outbound path sorts keys, and the request bodies for the two qwen3.5 models differ only in the model name, with the schema reaching the wire in declaration order. The grammar itself is any-order: qwen3.5:4b reproduces the required arrays exactly and qwen2.5:14b emits type first in 220/220 nodes, so the same grammar accepts both orderings. On a toy schema with no oneOf, no recursion and no plan vocabulary—and a worked example putting type first—qwen3.5-9b still alphabetises (5/5), and with three keys it places type in the middle, which excludes mere reversal. The behaviour is therefore a schema-mode output convention learned by qwen3.5-9b, harmless in itself, that is fatal only in contact with a oneOf discriminated by a type field.

We resist two tempting generalisations. Model size is not

the axis: qwen2.5:14b is larger than qwen3.5-9b and is unaffected. Nor is qwen3.5:4b “ignoring” the schema while the larger model obeys it—4b is demonstrably constrained, its optional-property use collapsing from 103 emissions to 0 on otherwise byte-identical prompts, and both models emit schema-valid JSON. What differs is an orthogonal ordering convention. The generalisable claim is therefore about schema design rather than about model scale: *a oneOf discriminated by a type field is unsafe under constrained decoding on any model that alphabetises keys in schema mode*, because the discriminator is then emitted after the fields that distinguish the branches. We state that at the branch counts actually tested—six and four, both of which fail. A two-branch union did not reproduce the effect in an aborted side probe (a toy move/wait union, $N=5$: qwen3.5-9b chose correctly 3/5 and opened objects with goals, qwen3.5:4b 4/5), which is ten calls and is offered only as a reason to scope the claim rather than as a result.

Remedies, none of which we have tested. The ladder localises the fault but does not supply a fix, and we are explicit that the two candidates below are untested. S3 is a diagnostic, not a proposal: a schema that can express one node type is unusable in deployment, because the system does not know before the operator speaks whether the answer is a movement, a formation, or a refusal. The first candidate is instead a *fixed envelope*: pin the top-level node to sequence and move the union inside its steps. The data point toward this rather than away from it—under S1 the model emits mapf 53 times nested inside a sequence while never emitting it at top level, and under S1-no-idle the degeneracy moves *into* sequence (43 of 65)—so the branch decision appears to be tractable once a container is already open. Unlike S3 this keeps the full grammar and is deployable, but we have not run it. The second candidate is renaming the discriminator so that it sorts before every content key (action rather than type, which precedes follower_ns, formation_id, goals, reason and steps). It follows directly from the mechanism and is a one-line change; it is also the obvious test to have run, and we did not run it. Both are stated as predictions the mechanism makes, and either could fail.

The deployment hazard is in the defaults, and the obvious fix makes it worse. Two parameters interact here. `_stream_and_parse()` tests `tool_calling_enabled` first and routes to the tool loop when it is set; the schema is applied only on the other branch, inside `_stream_plain()`. The two are therefore mutually exclusive in practice, and the code says so—it logs a warning when both are enabled, noting that `structured_output_enabled` will not constrain those turns. The declared defaults in `chat_server.py` are `tool_calling_enabled false` and `structured_output_enabled true`, while `orchestrator.yaml` ships both *true*. The deployed configuration thus avoids constrained decoding entirely, and only because tool calling happens to be switched on: the schema is enabled in the file but never reaches the model. The code defaults are the combination E2 measures at 11.4%, and an operator following the warning’s own advice—disable tool

calling, since it suppresses structured output anyway—moves the system from the unconstrained path onto exactly that combination. The hazard is conditional on model choice rather than universal, since `orchestrator.yaml` names `mistral-small13.1` rather than one of the models measured here; but a defaults pair that is safe only by accident, and that a documented remedy turns unsafe, is a configuration defect independent of which model is loaded.

Symbolic grounding is load-bearing for the small model and inert—or negative—for the large one. `qwen3.5:4b` degrades monotonically as scaffolding is removed, losing 9.1 points of `pass@5` by *coords-only*, with *wrong structure* rising $17 \rightarrow 24$ and *wrong robot-id set* $0 \rightarrow 3$: stripped of aliases and zones it starts mis-resolving which robots and which structure the request implies. `qwen3.5:9b` loses nothing (90.9% `pass@5` in all three grounding conditions) and *gains* 9.1 points of `pass@1` in both reduced conditions. Since *locations-only* differs from *full* only by the heuristics paragraph, that paragraph is a net negative for the larger model—it is prose advice about corridor deadlocks and room capacities, and the model plans better without it. This is a concrete answer to RQ2’s question of what the $\approx 9,700$ -token prompt buys: for the deployed model, the most expensive symbolic block is counterproductive, and for the small model it is nearly free while the *aliases and zones* carry the value.

The *unp.* column carries a second, independent effect that the pass rates hide. Trimming the map block monotonically reduces malformed JSON in both models: $16 \rightarrow 11 \rightarrow 11$ for `qwen3.5:4b` and $6 \rightarrow 1 \rightarrow 0$ for `qwen3.5:9b`, which for the larger model is the malformed-output failure mode disappearing entirely without any schema. This runs in the same direction as the heuristics result and is worth separating from it: a shorter prompt appears to buy *cleaner output* independently of whether it buys better plans. For `qwen3.5:4b` the two effects point opposite ways—it emits better-formed JSON on the reduced prompts and nonetheless plans worse—so output well-formedness and planning accuracy are not the same axis and a prompt-budget decision cannot be made on either alone. At five and six failures respectively these counts are small, and we report the direction rather than an effect size.

Reading with appropriate caution. Two cautions bound the above. The grounding effect on `qwen3.5:4b` rests on differences of 2–9 points at $N=5$ over 44 cases, which is enough to show the direction and not enough to bound the effect size; only the *coords-only* `pass@5` drop (-9.1) is comfortably larger than the per-cell noise visible elsewhere in the table. More importantly, `qwen3.5:9b`’s *locations-only* and *coords-only* cells produce identical aggregates (86.4%/90.9%). These are genuinely different prompts and their per-category results do differ (`mapf_explicit` 50.0/100.0 vs. 100.0/100.0; `formation_complex` 100.0/100.0 vs. 66.7/100.0), so the aggregate match is a coincidence at 44 cases rather than evidence that the two conditions are equivalent, and we read no effect into it. The schema result on `qwen3.5:9b` needs no such hedging: a 65.9-point drop with eight categories at exactly zero is not a sampling artifact.

What E2 does not measure. Sec. 5.4 proposed five factors;

two are reported here. Tool calling is deferred to E3. The offline harness cannot exercise it faithfully: `check_occupancy` subscribes to a live `LaserScan` and so requires the simulator; `verify_plan_execution_state` verifies a post-execution state that does not exist offline; the task tools are irrelevant to all 44 cases; and `get_robot_position/get_positions` return data the harness already injects into the prompt. Only two pure geometry helpers would carry signal, so an offline tool arm would weigh the full cost of the tool path against a fraction of its benefit. Standing up a fake-odometry publisher to feed the tools was considered and rejected: that plumbing would only transport poses the harness already holds, leaving the measurement unchanged. This reverses the intention stated in Sec. 5.4; the objection recorded there—that measuring the factor in E3 confounds planning mode with execution noise—remains valid and is the price of the change. Live pose context, goal auto-spread, and context budget are likewise not measured: the first is uninterpretable as a single rate because several cases exist *because of* their context, and the latter two are not on the harness code path at all (`goal_spread_enabled` is applied by `_postprocess_plan()` after the model returns, so E1 neither used nor could have used it).

One side effect of the schema factor is worth recording, since it bears on auto-spread from the other direction. `spread` is the only optional property in the plan schema, and constrained decoding suppresses it sharply: $103 \rightarrow 0$ emissions for `qwen3.5:4b`, $99 \rightarrow 0$ for `qwen3.5:9b`, $191 \rightarrow 9$ for `qwen2.5:14b`. Optional properties are not forbidden by the grammar—14b still emits nine—but they become rare enough to treat as unavailable. The capability survives only because `_validate_node()` infers spreading from goal arity when one centre is given for several robots; 4b’s `mapf_spread` score in fact rises under the schema. That inference is load-bearing in a way its own comment does not suggest, and removing it would silently disable the feature for any deployment using structured outputs.

[PENDING RESULTS: E3 — end-to-end mission completion by configuration, with LLM-call cost and guard/repair firing counts.]

[PENDING RESULTS: E3b — channel-1 decision confusion matrix and false-replan rate.]

7 Conclusion

[PENDING RESULTS: Full claim pending E3. The intended claim remains: a small typed plan grammar plus deterministic guards lets a modestly-sized locally-hosted model supervise a 20-robot fleet, and the gap between plan-level accuracy and mission-level success is where the engineering value sits—E3 is the experiment that can actually support that sentence.]

What E1 and E2 support is narrower but not nothing. First, bare planning accuracy without any of the system’s guards or repair loops sits at 68–77% `pass@1` across five locally-hosted 3.4–24B models under a realistic $\approx 9,700$ -token prompt—a fleet-supervision system built on a single unguarded plan call would fail roughly one command in three to one in four *regardless of which model is deployed*, which is the quantitative

case for the defence-in-depth design of Sec. 4.4 rather than an assumption behind it. Second, the five models’ failures are not interchangeable, and not even uniformly noisy: the smallest model’s errors look like sampling noise (pass@5 recovers most of the gap to a near-perfect pass@1), while two mid/large models repeat the *identical* validator complaint—a malformed `offsets_x/offsets_y` array in formation nodes—on retry after retry. That argues for model-specific schema-repair logic rather than a single fixed remediation budget applied uniformly. Third, escalation judgment (recognizing missing or ambiguous state and asking for help instead of acting) is weak and essentially flat across all five models independent of size or family, which is the strongest single piece of evidence in this data for keeping that decision outside the model rather than trying to prompt it better. Fourth, the categories that are hardest across the board—formation staging order, escalation judgment—are exactly the categories the plan grammar (Sec. 4.2) and the guard layer were built to catch downstream, which is a consistency check on the architecture rather than a result about it.

E2 adds a fifth point that we did not anticipate and that cuts against one of our own design choices. Schema-constrained decoding—listed above as the first line of defence—is not a free safety net. It repairs the failure mode it was built for only where that failure mode exists, does nothing where it does not, and on one of the three models tested under it destroys planning outright, by a mechanism (alphabetised keys meeting a `type-discriminated oneOf`) that has nothing to do with planning ability. We can locate the fault precisely—the schema ladder puts it in branch *selection*, not in recursion and not in filling a branch once chosen—but we stop short of claiming a fix: the two remedies the mechanism suggests, a fixed *sequence* envelope and renaming the discriminator so it sorts first, are both untested. The wider lesson for defence-in-depth is that a guard is only deterministic from the system’s side; from the model’s side it is another distribution shift, and it has to be measured per model rather than assumed. Whether the remaining guards actually close the plan-to-mission gap in a running fleet, and whether it holds under tool calling instead of constrained decoding, is what E3 is for.

Code availability

https://github.com/innopolis-robotics-society/llm_swarm

References

- [1] R. Stern et al., “Multi-agent pathfinding: Definitions, variants, and benchmarks,” SoCS, 2019.
- [2] G. Sharon et al., “Conflict-based search for optimal multi-agent pathfinding,” *Artificial Intelligence*, 2015.
- [3] H. Ma et al., “Searching with consistent prioritization for multi-agent path finding,” AAAI, 2019.
- [4] J. Li et al., “MAPF-LNS2: Fast repairing for multi-agent path finding via large neighborhood search,” AAAI, 2022.
- [5] C. Jiang, Z. Chen, and Y. Guo, “Multi-robot formation control: a comparison between model-based and learning-based methods,” *J. Control Decis.*, vol. 7, no. 1, pp. 90–108, 2020.
- [6] E. Debie, K. Kasmarik, and M. Garratt, “Swarm robotics: A survey from a multi-tasking perspective,” *ACM Comput. Surv.*, vol. 56, no. 2, 2023.
- [7] H. Wang, M. Yue, X. Sun, and X. Zhao, “Cooperative formation control strategy for multi-robot system based on APF algorithm and sliding mode estimator,” *Robot. Auton. Syst.*, vol. 193, p. 105075, 2025.
- [8] S. Macenski et al., “The Marathon 2: A navigation system,” IROS, 2020.
- [9] D. V. Lu et al., “Layered costmaps for context-sensitive navigation,” IROS, 2014.
- [10] M. Colledanchise and P. Ögren, *Behavior Trees in Robotics and AI*, CRC Press, 2018.
- [11] D. Faconti, “BehaviorTree.CPP” <https://www.behaviortree.dev>
- [12] M. Ahn et al., “Do as I can, not as I say: Grounding language in robotic affordances,” CoRL, 2022.
- [13] J. Liang et al., “Code as policies: Language model programs for embodied control,” ICRA, 2023.
- [14] I. Singh et al., “ProgPrompt: Generating situated robot task plans using large language models,” ICRA, 2023.
- [15] W. Huang et al., “Inner monologue: Embodied reasoning through planning with language models,” CoRL, 2022.
- [16] Y. Maruyama, S. Kato, and T. Azumi, “Exploring the performance of ROS2,” EMSOFT, 2016.
- [17] T. Kronauer, J. Pohlmann, M. Matthe, T. Smejkal, and G. Fettweis, “Latency analysis of ROS2 multi-node systems,” 2023.
- [18] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Sci. Robot.*, vol. 7, no. 66, 2022.
- [19] R. Vaughan, “Massively multi-robot simulation in Stage,” *Swarm Intell.*, vol. 2, pp. 189–208, 2008.
- [20] N. Koenig and A. Howard, “Design and use paradigms for Gazebo, an open-source multi-robot simulator,” IROS, 2004.
- [21] M. Iovino, E. Scukins, J. Styruud, P. Ögren, and C. Smith, “A survey of behavior trees in robotics and AI,” *Robot. Auton. Syst.*, vol. 154, 2022.
- [22] T. Schick et al., “Toolformer: Language models can teach themselves to use tools,” NeurIPS, 2023.
- [23] F. Zeng et al., “Large language models for robotics: A survey,” 2023.
- [24] A. Brohan et al., “RT-2: Vision-language-action models transfer web knowledge to robotic control,” arXiv:2307.15818, 2023.